

280 mondat — záróvizsga beszédvázlat

EKKE Programtervező Informatikus BSc · Záróvizsga · 14 tétel × 2 alpont × 10 mondat

280 mondat — záróvizsga beszédvázlat

Alpontonként 10 összefüggő, kimondható mondat. A hivatalos kérdés minden topicját érinti, sorrendben olvasva folyamatos beszéd.

1.a Bevezetés az informatikába

1. Az **információ** új ismereteket hordozó jelek tartalmi befogadása, ami cselekvésre késztet; az **adat** ennek a hordozója, önmagában jelentés nélküli.
 2. Az információ útja: **adó** → **kódolás** → **csatorna (zaj)** → **dekódolás** → **vevő**, ezért a kódolás-dekódolás minőség kulcsfontosságú.
 3. Az információ mértékegysége a **bit**: két egyenlően valószínű jel egyikének kiválasztásához tartozó információmennyiség.
 4. Az **entrópia** a rendszer bizonytalanságát méri, képlete $H = - \sum p_i \cdot \log_2 p_i$, ahol a valószínűségek összege 1.
 5. Az **átlagos kódhossz** $\sum p_i \cdot h_i$, a **kódolási hatásfok** ennek és az entrópiának a hányadosa — jó kódolásnál közel 1.
 6. A **Shannon-Fano** eljárás gyakoriság szerint csökkenően rendez, közel egyenlő részekre osztja a listát és 0/1-et rendel, rekurzívan halad.
 7. A **Huffman-kódolás** összevonja a két legkisebb valószínűségű jelet és visszafelé olvassa ki a kódokat — optimális prefix kódot ad.
 8. **Fixpontos számábrázolás**: bináris, a legmagasabb bit az előjel, a negatív számokat kettes komplementessel ábrázoljuk.
 9. **Lebegőpontos (IEEE 754, 32 bit)**: 1 előjelbit + 8 karakterisztika (eltolásos +127) + 23 mantissza (implicit bittel).
 10. A **karakterkódolás** ASCII (128 karakter) vagy Unicode (UTF-8) — a gép számára teszi értelmezhetővé a szöveget.
-

1.b Elsőrendű logika

1. Az elsőrendű logika **paraméteres állításokkal (predikátumokkal)** dolgozik, amik termeken értelmezettek — termek: konstansok, változók, függvények.
 2. Két **kvantor** van: $\forall$ ("minden x-re") és $\exists$ ("van olyan x"); a nyelv egy $\langle \text{Var}, \text{Pr}, \text{Fn} \rangle$ formális hármas.
 3. A **szintaxis** a helyes formulák nyelvtana, a **szemantika** az értelmezés: az interpretáció definiálja a **domaint** és minden szimbólumhoz értéket rendel.
 4. Egy formula **tautológia** (minden interpretációban igaz), **kontradikció** (mindig hamis) vagy **kielégíthető** (legalább egyben igaz).
 5. A **KNF-re hozás 6 lépése**: implikáció eltüntetése, változótiszta alak, prenexizálás, Skolemizálás, KNF-re hozás disztributivitással, klózokra bontás.
 6. **Skolemizálás**: az $\exists x$ kvantort eltüntetjük, és x helyébe új **Skolem-függvényt** írunk, amelynek paraméterei az előtte álló $\forall$ kvantorok változói; $\forall$ nélkül Skolem-konstans.
 7. A **rezolúció** cáfoló eljárás: a bizonyítandó állítás tagadásából indulunk, és levezetjük az üres klózt (UNSAT).
 8. Az **unifikáció** karakterről karakterre egyformává tesz két literált — változót konstanssal/függvénnyel illeszt, az **occur check** szerint $x \rightarrow f(x)$ tilos.
 9. A **lineáris rezolúció** mindig az előző rezolvensre épít (teljes); az **SLD rezolúció** célirányosabb, és **Horn-klózok** esetén teljes.
 10. A **Prolog** az első logikai programozási nyelv: Horn-klózokkal, SLD-rezolúcióval dönti el, hogy egy célformula logikai következménye-e a tényeknek és szabályoknak.
-

2.a Magasszintű programozási nyelvek I

1. A **tömb és lista** homogén, véletlen elérésű, folytonos adatszerkezet — a tömb fix méretű, a `List<T>` dinamikusan bővül.
 2. A **rekord** heterogén mezőket tartalmaz; C#-ban a **class** referenciátípus (heap, GC), a **struct** értéktípus (stack, érték szerint másolódik).
 3. A **felsorolásos típus (enum)** megnevezett konstansok halmaza, alpból int-re fordul.
 4. A **metódus** szignatúrával rendelkezik (név + paramétertípusok), lehet **static** (osztálysztintű) vagy példányszintű, és **túlterhelhető**.
 5. **Paraméterátadás módjai**: érték szerint (alap eset), **ref** (referencia, be+ki), **out** (kötelező kimenet), **in** (csak olvasható), **params** (változó paraméterszám tömbként).
 6. Nyelvek **fordítási megoldásai**: AOT (C, Rust — előre gépi kódra), értelmezett (Bash), vagy **JIT** (Java JVM, C# CLR — futáskor fordul).
 7. A **.NET** fő összetevői: **CLR** (futtatás, JIT, GC), **BCL** (alapsztálykönyvtár), **IL** (közbenső kód) és nyelvi compilerek.
 8. C# forrásból **IL** lesz, amit a CLR futás közben gépi kódra fordít, és kezeli a memóriát, típusbiztonságot, kivételeket.
 9. Hasonló **JVM** Javához (bytecode + JIT), és a skála az AOT-tól (Go, Rust) a tisztán interpretáltig (CPython) terjed.
 10. A virtuális gépes nyelvek **platformfüggetlenséget** adnak — ugyanaz a kód több OS-en fut ugyanazzal a runtime-mal.
-

2.b Adatszerkezetek és algoritmusok — programozási tételek

1. Az **algoritmus** véges, egyértelmű, általános, hatékony lépéssorozat a feladat megoldására.
 2. **Megadási módok**: szöveges, pszeudokód, folyamatábra, struktogram (Nassi-Shneiderman), Jackson-diagram.
 3. A **strukturált algoritmus** három alapszerkezete: **szekvencia, szelekció, iteráció** — goto nélkül.
 4. **Sorozathoz elemi értéket** rendelő tételek: eldöntés, kiválasztás, lineáris keresés, megszámlálás, maximumkeresés, összegzés.
 5. **Sorozathoz sorozatot** rendelők: másolás (transzformáció), kiválogatás, szétválogatás; **több sorozathoz egyet**: unió, metszet, összefuttatás.
 6. **Rendezések**: buborék/beszúrásos/kiválasztásos $O(n^2)$; **quicksort/mergesort/heapsort** $O(n \log n)$ — a quicksort legrosszabb esete $O(n^2)$.
 7. A **mergesort** stabil és mindig $O(n \log n)$, de $O(n)$ extra memória kell; a quicksort átlagosan gyors, in-place, de instabil.
 8. A **visszalépéses keresés (backtracking)** rekurzív próbálkozás: zsákutcánál visszalépés és másik opció — N királynő, sudoku, gráfszínezés.
 9. A **programozási tételek** átültethetők különböző homogén adatszerkezetekre: tömbön, láncolt listán, halmazon, fán értelmezhetők.
 10. A **halmaz adatszerkezet** konstrukciói: rendezetlen lista ($O(n)$), rendezett tömb bináris kereséssel ($O(\log n)$), karakterisztikus függvény bit-vektorral ($O(1)$), hash-tábla, fa.
-

3.a Adatbázisrendszerek I — modellek

1. Három klasszikus adatbázis-modell van: **hierarchikus** (fa, 1 szülő), **hálós** (több szülő, M:N), és a domináns **relációs modell**.
 2. A relációs modell **táblákban** tárol adatot, a kapcsolatokat **kulcsok** és **idegen kulcsok** reprezentálják, halmazalapú SQL-lel kérdezhető.
 3. **Kulcsfajták**: superkulcs (mindent meghatároz), candidate (minimális superkulcs), **primary** (kiválasztott), **foreign** (másik tábla PK-jára mutat).
 4. **Kapcsolat-típusok**: 1:1 (személy ↔ útlevelel), 1:N (osztály ↔ tanuló), M:N (hallgató ↔ tantárgy) — utóbbi **kapcsolótáblával** valósul meg.
 5. **Anomáliák** normalizálatlan táblákban: beszűrési (új adat csak másikkal vihető be), módosítási (több helyen kell javítani), törlési (más adat törlése értékes infót töröl).
 6. A **funkcionális függőség** $X \rightarrow Y$: ha X megegyezik két sorban, Y is megegyezik; a **transzitivitás** miatt $X \rightarrow Y$ és $Y \rightarrow Z$ -ből $X \rightarrow Z$.
 7. **1NF**: minden cella **atomi** (nem összetett, nem ismétlődő).
 8. **2NF**: 1NF + minden nem-kulcs attribútum teljesen függ az **összes** elsődleges kulcstól (összetett kulcsnál releváns).
 9. **3NF**: 2NF + nincs **transzitiv** függőség a nem-kulcs attribútumok között.
 10. A **BCNF** a 3NF szigorúbb változata (minden FF bal oldala superkulcs); gyakorlatban a 3NF/BCNF a célzott szint.
-

3.b Dinamikus adatszerkezetek

1. A hatékonyságot **algoritmizálási** (paradigma, korai kilépés) és **adatkonstruksiós** (cache-barát, hely-idő tradeoff) döntések befolyásolják.
 2. A **verem (Stack)** LIFO szerkezet **push/pop/peek** műveletekkel; használat: hívási lánc, kifejezés-kiértékelés, undo.
 3. A **sor (Queue)** FIFO szerkezet **enqueue/dequeue** -val; használat: ütemezés, BFS gráfbejárás, üzenetsorok.
 4. A **láncolt lista** csomópontokból + mutatókból áll: beszúrás/törlés ismert pozíción $O(1)$, hozzáférés index szerint $O(n)$ — szemben a tömbbel.
 5. A **hash-tábla** hash-függvénnyel képez kulcsot indexre, átlag $O(1)$ műveletek; ütközéskezelés **láncolással** vagy **nyílt címzéssel**.
 6. **Kereső algoritmusok**: lineáris $O(n)$, bináris (rendezett) $O(\log n)$, hash $O(1)$, kiegyensúlyozott fa $O(\log n)$.
 7. A **programozási tételek** átültethetők: tömbön index-iteráció, láncolt listán mutatóval, fán bejárások keretében.
 8. A **rekurzió** önhívás bázisesettel + rekurzív hívással; klasszikus: faktoriális, Fibonacci, Hanoi, fa-bejárások.
 9. **Rekurzió vs iteráció**: a rekurzió a hívási vermet használja (stack overflow!), az iteráció konstans memóriájú — minden rekurzív átírható iteratívra.
 10. A **fa** csomópontokból + gyökérből áll; a **BST**-n bal részfa < csomópont < jobb részfa, bejárások: **preorder**, **inorder** (rendezett kimenet), **postorder**, BFS.
-

4.a Magasszintű programozási nyelvek I — alapok

- 1. Primitív típusok:** byte/short/int (32)/long (64), float/double/decimal, bool, char (Unicode), és a `string` mint immutable referenciatípus.
 - 2. A változó** memóriaterület + név + típus; a `const` fordításkor rögzül, a `readonly` csak konstruktorban kaphat értéket, a `literál` beégetett érték.
 - 3. Operátorok:** aritmetikai, összehasonlító, logikai rövidzárú (`&&` , `||`), bitenkénti, ternáris, null-coalescing (`??`).
 - 4. Szelekciós szerkezetek:** `if/else if/else` , `switch` (modern C#-ban pattern matchinggel).
 - 5. Ciklusok:** `while` (elől), `do-while` (hátul), `for` (számláló), `foreach` (IEnumerable-n); vezérlés: `break` , `continue` , `return` .
 - 6. Értéktípusok** (struct, int) a `stack`-en, érték szerint másolódnak; **referenciatípusok** (class) a `heap`-en, csak referenciát másolunk.
 - 7. A stack** gyors LIFO és korlátos méretű (stack overflow); a `heap` dinamikus, a `GC` kezeli; a `boxing/unboxing` lassú, kerülendő.
 - 8. A változó hatásköre** mondja meg, hol látszik (block, metódus, osztály), az **élettartama** azt, mikor él (lokális, mező, static).
 - 9. Nyelvgenerációk:** 1GL gépi kód, 2GL assembly, 3GL magas szintű (C, C#, Java), 4GL DSL (SQL), 5GL logikai (Prolog).
 - 10. Paradigma-családok:** imperatív (hogyan), funkcionális (függvény-kompozíció, immutable), deklaratív (mit — SQL), objektum-orientált, logikai.
-

4.b Operációs rendszerek

1. Az operációs rendszer a **hardver és az alkalmazások közötti réteg**, ami erőforrást menedzsel és API-t nyújt.
 2. A **kernel** privilegizált módban fut és közvetlenül a hardverrel beszél — lehet **monolitikus** (Linux), **mikrokernel** (Minix), vagy **hibrid** (Windows NT).
 3. A **processz** saját címtérrel + PID-del + állapottal rendelkezik; a **szál** processzen belüli, közös memóriájú végrehajtási egység gyors kontextusváltással.
 4. A **virtualizáció** lehet teljes (külön OS, hypervisor), para- (vendég tud róla), vagy **OS-szintű** (konténer, közös kernel — Docker).
 5. **Fájlrendszerek** hierarchikus szervezést, attribútumokat, ACL-eket, lock-olást, **journalingot** nyújtanak — NTFS, ext4, APFS, ZFS, FAT32.
 6. **RAID** szintek: 0 (striping, nincs redundancia), 1 (mirror), 5 (striping + 1 paritás), 6 (2 paritás), 10 (mirror + stripe).
 7. **Átírányítások** (`>` , `>>` , `<` , `|`) és **szűrők** (`grep` , `sed` , `awk` , `sort` , `uniq`) láncolnak parancsokat Unix-filozófiában.
 8. **Jogosultságok**: Linuxon owner/group/others × r/w/x (`chmod 755`); Windowson részletes **ACL** Allow/Deny szabályokkal.
 9. **Processz-állapotok**: new → ready → running → waiting → terminated; ütemezők: FCFS, SJF, Round Robin, priorítás, multilevel feedback.
 10. **Szignálok** (SIGTERM, SIGKILL, SIGINT) aszinkron értesítések; **backup**: full/inkrementális/differenciális (3-2-1 szabály); **shell-script** automatizál parancsokat shebanggel és vezérléssel.
-

5.a OOP alapok

1. Az **OOP négy alapelve**: egységbe záras, adatrejtés, öröklődés, polimorfizmus.
 2. Az **osztály** sablon az objektumokhoz: **mezők** az adatot tárolják, **metódusok** a viselkedést, a **konstruktor** az inicializálást.
 3. **Láthatóság-szabályozók**: **private** (csak az osztály), **public**, **protected** (+ leszármazottak), **internal** (assembly-n belül).
 4. A **property** C# specifikus: getter/setter mögött metódus fut, ezért validáció és invariáns érvényesíthető hozzáféréskor.
 5. **Konténerosztályok** a **System.Collections.Generic** -ben: **List<T>**, **Dictionary<K,V>**, **Stack<T>**, **Queue<T>**, **HashSet<T>** — mind generikus, típusbiztos.
 6. Az **indexelő** (**this[int i]**) lehetővé teszi, hogy az objektum **tömbszerűen indexelhető** legyen saját getter/setterrel.
 7. **Static (osztályszintű)** tag az osztályhoz tartozik (Math.Sqrt), nem példányhoz; a **static konstruktor** egyszer fut, az osztály első használatakor.
 8. **Névterek (namespace)** logikailag csoportosítják a típusokat és kerülik a névütközést — pl. **System.IO**, **System.Linq**.
 9. **Bővítő metódus**: static osztály static metódusa, első paraméter elé **this** — meglévő típushoz adunk metódust forrásmódosítás nélkül; a LINQ erre épül.
 10. **Operator overloading**: saját operátorok definiálhatók (csak static); az **==** és **≠** párban, és ha **==** -t definiálsz, **Equals** és **GetHashCode** is felülbírálandó.
-

5.b Architektúrák — cache + I/O

1. A **cache** kis, gyors átmeneti tároló a CPU és a memória közt — **cache hit** esetén nincs memóriához fordulás, **miss** esetén várni kell.
 2. A cache működésének alapja a **lokalitási elv: időbeli** (a most használt adatot újra használjuk) és **térbeli** (a szomszéd is jön).
 3. **Memória-hierarchia**: regiszter → L1 (32-64 KB, 3-5 ciklus) → L2 → L3 → RAM (100-200 ciklus) → SSD/HDD; a cache a CPU és a RAM közé ékelődik.
 4. A cache **sorokra (cache line, ~64 bájt)** van osztva: tag (cím), adatblokk, **valid bit**, **dirty bit**.
 5. **Elérési változatok**: direkt leképezésű (1 hely, gyors, ütközhet), fully associative (bárhova, drága), n-way set associative (n hely, kompromisszum).
 6. A **cím három részre** bomlik: **tag** (blokkot azonosít), **index** (melyik sor/halmaz), **offset** (blokkon belüli bájt).
 7. **Write-through**: minden módosítás azonnal a memóriába is megy (egyszerű, lassú); **write-back**: csak cache-be + dirty bit, sor kiváltásakor írunk vissza.
 8. A **valid bit** jelzi, hogy a sor adata érvényes-e (induláskor 0); a **dirty bit** jelzi, hogy módosult-e (kiváltás előtt visszaírás kell).
 9. **I/O módok**: programmed I/O (polling, CPU-igényes), interrupt-vezérelt (periféria szól), **DMA** (közvetlen memória-periféria adatmozgás, CPU csak indít).
 10. **Perifériák**: PC-n USB billentyűzet/monitor/SSD; mobilon érintőképernyő, accelerometer, GPS, kamera; robotikán LIDAR, IMU, enkóder + GPIO/PWM/I²C/SPI vezérlés.
-

6.a Numerikus matematika

- 1. Hibák négy típusa:** kiküszöbölhetetlen input-hibák, modell-hiba, módszer-hiba, gépi (kerekítési) hibák.
 - 2. Abszolút hiba** $|a - a_0|$, **relatív hiba** $(a - a_0)/a_0$, **hibaterjedés** $\Delta y \approx |f'(x)| \cdot \Delta x$ — a relatív hiba és a biztos számjegyek között logaritmikus kapcsolat van.
 - 3. Nemlineáris egyenletekre** ($f(x) = 0$) az **intervallumfelező eljárás** biztos, de lassú; feltétele: $f(a) \cdot f(b) < 0$ előjelváltás.
 - 4. A Newton-Raphson** érintő-módszer **kvadratis konvergenciát** ad: $x_{n+1} = x_n - f(x_n)/f'(x_n)$; deriválhatóság és jó kezdőérték kell.
 - 5. Numerikus integrálás:** téglalap-módszer nulladfokú, **trapéz** elsőfokú, **Simpson** másodfokú interpoláción alapul — egyre pontosabb.
 - 6. A Simpson-formula** súlyozása **1-4-2-4-2- ... -4-1** mintát ad, $(h/3)$ szorzóval — feltétele páros n .
 - 7. Lineáris egyenletrendszereket Gauss-eliminációval** oldunk: kiterjesztett mátrixot felső háromszögre alakítjuk elemi sorokvivalens átalakításokkal, visszahelyettesítünk.
 - 8. Polinomok kiértékelésére** a **Horner-módszer** szorzás-összeadássá rendezi a kifejezést (nincs hatványozás); a **Taylor-polinom** egy a pont körül polinomi sorral közelít.
 - 9. Interpoláció:** ismert pontokra polinomot illeszt — a **Lagrange** minden új pontnál újraszámol, a **Newton-féle (osztott differenciás)** inkrementális, csak az új tagot adjuk hozzá.
 - 10. A legkisebb négyzetek módszere** pontthalmazhoz a legjobban illeszkedő egyenest keresi az eltérések négyzetösszegét minimalizálva — nem interpoláció.
-

6.b Operációs rendszerek — funkciók

1. Az OS **fő funkciói**: erőforrás-menedzsment, processz/szál kezelés, memóriakezelés, fájlrendszer, I/O, biztonság, hálózat, felhasználói felület.
 2. A **kernel** privilegizált módban fut, a processzek user módban; a kontextusváltás közöttük **system call**-okon át történik.
 3. A **processz** saját címtérrel + PID-del rendelkezik; a **szál** processzen belüli, közös memóriájú végrehajtási egység gyors váltással.
 4. **Ütemezési algoritmusok**: FCFS, SJF, Round Robin (időszelet), prioritás-alapú, multilevel feedback queue — méltányosság és válaszidő közti tradeoff.
 5. **Virtualizáció**: teljes (külön OS hypervisorral) vagy **konténer** (közös kernel, izolált processz-tér, pl. Docker).
 6. A **fájlrendszer** hierarchikus szervezést, jogosultság-vezérlést, lock-olást, **journalingot** nyújt — utóbbi crash után konzisztens állapotot ad.
 7. **Tipikus fájlrendszerek**: NTFS (Windows), ext4 (Linux), APFS (macOS), ZFS/Btrfs (modern, snapshot), FAT32 (kompatibilitás).
 8. **Hibatűrő diszk-rendszerek (RAID)**: RAID 1 mirror, RAID 5 striping + 1 paritás (1 lemez-hibát tűr), RAID 6 (2), RAID 10 (mirror + stripe).
 9. **Jogosultsági rendszer**: **autentikáció** (ki vagyok) + **autorizáció** (mit tehetek); Linuxon UID/GID + bitek, Windowson ACL.
 10. **Backup szintek**: full (minden), inkrementális (utolsó óta), differenciális (utolsó full óta) — a **3-2-1 szabály** szerint 3 másolat, 2 médium, 1 off-site.
-

7.a OOP — öröklődés

1. Az **öröklődés** új osztály egy meglévő alapján: a leszármazott örökli a védett és publikus tagokat, kibővítheti, felülbírálhatja.
 2. C#-ban **egyszeres osztály-öröklődés** + tetszőleges interface implementálható; a **sealed** kulcsszó megtiltja a további öröklődést.
 3. **Korai kötés** fordításkor a deklarált típus alapján; **késői kötés** futáskor az objektum tényleges típusa alapján — **virtual** + **override** kell hozzá.
 4. A **virtual** metódusokhoz az osztály egy **DMT-t (vtable)** épít, és ebből oldódik fel a hívás — ez működteti a polimorfizmust.
 5. A **konstruktor** az osztály nevével megegyezik, nincs visszatérési típusa; lehet több (overload), és **: this(...)** szintaxissal hívható másik saját konstruktor.
 6. Származtatott osztály létrehozásakor **először az ős konstruktora fut le**, majd a leszármazotté — explicit **: base(...)** hívással irányítható.
 7. A **static (osztályszintű) konstruktor** egyszer fut, automatikusan, az osztály első használata előtt — a static mezőket inicializálja.
 8. **Típuskompatibilitás**: leszármazott implicit **upcastolható** ősre; **downcast** explicit cast vagy **as / is** operátorral, és kockázatos.
 9. Az **object** minden típus végső őse — 4 örökölt metódusa: **ToString**, **Equals**, **GetHashCode**, **GetType**.
 10. A **statikus osztály** nem példányosítható (csak static tagok, pl. Math); a **dynamic** kulcsszó (DLR-rel) futáskor oldja fel a típusokat — rugalmas, de elveszti a fordítóidejű biztonságot.
-

7.b Adatbázis I — SQL

1. Az **SQL** szabványos, deklaratív lekérdező nyelv relációs adatbázisokhoz; al-nyelvei a **DDL** (séma), **DML** (adatok), **DCL** (jogok), **TCL** (tranzakció).
 2. **Relációsémát** `CREATE TABLE` -el definiálunk; megszorítások: `PRIMARY KEY` , `FOREIGN KEY ... REFERENCES` , `NOT NULL` , `UNIQUE` , `CHECK` , `DEFAULT` .
 3. Az **index** B-fa alapú segédstruktúra, ami **felgyorsítja a keresést** — gyors SELECT, lassabb INSERT/UPDATE; a PK automatikusan indexelt.
 4. **Táblák módosítása:** `ALTER TABLE ADD/MODIFY/DROP` , `DROP TABLE` .
 5. A **SELECT alapforma:** `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT` ; a logikai kiértékelés sorrendje FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.
 6. **Aggregáló függvények:** `COUNT` , `SUM` , `AVG` , `MIN` , `MAX` ; a `HAVING` csoportokon, a `WHERE` egyedi sorokon szűr.
 7. **JOIN típusok:** INNER (csak párosak), LEFT (bal min., jobbon NULL), RIGHT, FULL, CROSS (Descartes-szorzat) — egyenlőségi feltétellel.
 8. **Beágyazott lekérdezés (subquery):** skalár, `IN` / `NOT IN` , `EXISTS` , korrelált — a belső kérdés a külső sor egy oszlopára hivatkozik.
 9. **GRANT** és **REVOKE** adja és veszi vissza a privilégiumokat; a **role** (`CREATE ROLE`) jogosultság-csoport, felhasználókhoz rendelhető.
 10. A **tranzakció ACID tulajdonságokkal** rendelkezik (atomosság, konzisztencia, izoláció, tartósság); `COMMIT` véglegesít, `ROLLBACK` visszagörget, **DDL implicit COMMIT-tal** jár.
-

8.a OOP haladó — abstract, generic, lambda

1. **Absztrakt metódus** (`abstract` , nincs törzs) még meg nem írt műveletet jelöl; ha van benne, az **osztály is abstract**, a gyerek kötelezően override-olja.
 2. Az **interface** nyelvi elem (nem osztály), ami **szerződést** definiál — metódus- és property-szignatúrákat tartalmazhat, mezőt és konstruktort nem.
 3. **Egyetlen ős-osztály + tetszőleges interface** — így C# megoldja a többszörös öröklődés problémáját; az interface más interface-ből származhat.
 4. A **kivételkezelés filozófiája**: a program normál módban fut, hiba esetén hiba módba lép, **visszaugrál a hívási láncon**, ha senki sem javítja, leáll.
 5. **try-catch-finally**: `try` a kritikus kód, `catch(Típus)` típus szerint kezel (több catch fentről le, speciális előrébb), `finally` mindig lefut — erőforrás-felszabadításhoz.
 6. **Generikusok** típussal paraméterezhető osztályok/metódusok (`List<T>` , `Dictionary<K,V>`) — általános viselkedés definiálható kódDuplikáció nélkül.
 7. A **delegate** metódusreferencia (funkciómutató); beépítettek: **Action** (eljárás), **Func** (függvény visszatéréssel), **Predicate** (1 paraméter, bool).
 8. Az **event** több metódust köthet egy eseményhez, és hívásra mind lefut — az **anonim delegate** egyszeri callbackként használható.
 9. A **lambda kifejezés** tömör függvény szintaxis (`n ⇒ n.Contains('i')`); a **LINQ** teljes egészében erre épül.
 10. A LINQ bővítő metódusokat ad (`Where` , `Select` , `GroupBy` , `Join`) `IEnumerable`-re, és **összefűzhető láncolható** lekérdezéseket épít — hasonló mint a Java Streams-ben és Python comprehensionokban.
-

8.b Fordítóprogramok

1. A **fordítóprogram** forrásnyelvi szövegből tárgykódot állít elő; a **compiler** magas szintű kódot gépi kódra fordít, az **interpreter** értelmezi futás közben.
 2. A **compiler felépítése**: input-handler → lexikális → szintaktikai → szemantikai elemző → belső reprezentáció → kódgenerátor → optimalizáló → kód-kezelő → tárgyprogram.
 3. A **lexikális elemző** karaktersorozatból **tokeneket** állít elő és reguláris (Chomsky-3) nyelvet használ — egy **lexikális hiba** ismeretlen token (pl. `int 2ab`).
 4. A **szimbólumtábla** a programban előforduló szimbólumok nevét és attribútumait (típus, cím, sorszám) tárolja — gyakran láncolt listával.
 5. A **reguláris nyelvek (Chomsky-3)** szabályai $A \rightarrow a$ vagy $A \rightarrow Ba$ alakúak, egy irányban épülnek — lexikális elemzésre alkalmasak.
 6. A **szintaktikai elemzők** eldöntik, hogy a program nyelvtanilag helyes-e, és **levezetési fát** állítanak elő.
 7. A **rekurzív leszállás** minden szimbólumhoz eljárást rendel, amik sorban hívják egymást — egyszerű, de nem generálható, nem elterjedt.
 8. Az **LR(k)** elemző bottom-up, az inputból indul a kezdőszimbólum felé; az **LL(k)** top-down, a kezdőszimbólumból indul, balról jobbra építi a fát.
 9. A **táblázatos elemző** állapot-hármassal $(ay\#, Xa\#, v)$ dolgozik: input maradéka, verem (mondatforma), alkalmazott szabályok — Első/Követő halmazok döntenek a helyettesítést.
 10. A **szemantikai elemzés** statikus tulajdonságokat vizsgál (típuskompatibilitás, deklaráció, hatáskör, túlterhelés-egyértelműség), amik környezetfüggetlen nyelvtannal nem írhatók le.
-

9.a Formális nyelvek

1. Az **ábécé** véges, nem üres jelhalmaz; a **szó** az ábécé jeleiből alkotott véges sorozat (ϵ az üres szó); a **formális nyelv** az ábécé fölötti szavak részhalmaza.
 2. **Műveletek szavakkal: konkatenáció** (asszociatív, nem kommutatív, neutrális ϵ), hatványozás, tükrözés (palindrom); nyelvekkel: komplexus szorzás, hatvány.
 3. **Szintaxisleíró eszközök**: a **szintaxisdiagram** vizuális (terminális/nemterminális téglalapok), az **EBNF** karakteres jelölés programnyelvekhez.
 4. A **generatív grammatika** $G(V, W, S, P)$ formális négyes: V terminálisok, W nemterminálisok ($V \cap W = \emptyset$), S kezdőszimbólum, P szabályok.
 5. A **Chomsky-hierarchia** 0-tól 3-ig szigorodik: 0 mondszerkezetű (Turing), 1 környezetfüggő (lin. korl. Turing), 2 környezetfüggetlen (verem-automata), 3 reguláris (véges automata).
 6. A **levezetési fa** környezetfüggetlen nyelvtannal dönti el, hogy egy szó generálható-e — **kanonikus levezetésnél** mindig a legbaloldalibb nemterminálist helyettesítjük tovább.
 7. A **véges automata** (K, V, δ, q_0, F) : nincs output, csak olvasó-fej; fix n lépés után megáll, akkor fogad el, ha elfogadó állapotban van.
 8. A **verem-automata** $(K, V, W, \delta, q_0, z_0, F)$: van veremje, ahova szót írhat, δ -be ϵ is mehet input gyanánt — **nem biztos, hogy megáll**.
 9. A **Turing-gép** $(K, V, W, \delta, q_0, B, F)$ az input szalagra is írhat, mindkét irányban végtelen; akkor áll meg, ha δ nincs értelmezve — változatai (több szalagú, lin. korlátolt) ekvivalensek.
 10. **Automaták és grammatikák** kapcsolata: a grammatika **generál**, az automata **felismer** — a Chomsky-osztály és az automata-típus egy-egyértelműen összepárosul.
-

9.b Rendszerfejlesztés technológiája

1. A szoftver **egyedi** (ismert megrendelő) vagy **dobozos** (ismeretlen vásárlók); az **életciklus** a módszertanok által meghatározott — cél a magas minőség minimális költséggel.
 2. **9 állomás**: új igény → követelményspec → rendszerjavaslat → rendszerspecifikáció → logikai+fizikai tervezés → implementáció → tesztelés → rendszerátadás → üzemeltetés.
 3. A **követelményspecifikáció** alapja **riportok** (szabad és irányított); tartalmazza a jelenlegi helyzetet, vágyálomrendszert, jogszabályi háttérrel, fogalomszótárt.
 4. A **funkcionális specifikáció** felhasználói szemszögből írja le a rendszert, a **nagyvonalú rendszerterv** fejlesztői szemszögből.
 5. Modern módszertanok **iteratívak**: a tervezés-implementáció-tesztelés ciklusok rövidek, minden iteráció szállíthat valamit.
 6. A **feladatkövetés** (Kanban/Scrum board) átláthatóvá teszi a feladatokat — csökkenti az állapotjelentés szükségességét.
 7. A **verziókövetés** (Git, SVN) a változásokat követi (ki, mikor, mit), támogat branchet, conflictnél értesít — a fejlesztés kulcseszköze.
 8. **Tesztelési technikák**: feketedobozos (csak spec), fehérdobozos (forráskód), szürkedobozos (részleges) — kódsorokat, elágazásokat, metódusokat tesztelünk.
 9. **Tesztelés szintjei**: unit (metódus), modul, integrációs, rendszerteszt, átadás-átvételi (alfa, béta, felhasználói, üzemeltetői).
 10. A **regressziós teszt** minden változtatás után **az összes unit-tesztet** lefuttatja — biztosítja, hogy nem rontottuk el a már működőt.
-

10.a Hálózati architektúrák

1. A **csomagkapcsolt hálózatok** az üzenetet csomagokra bontják, mindegyik fejléccel megy, különböző útvonalakon, és a sorrendet a célnak kell helyreraknia.
 2. Az **OSI 7 rétegű** elméleti modell: fizikai, adatkapcsolati, hálózati, szállítási, viszony, megjelenítési, alkalmazási — az oktatás referenciája.
 3. A **TCP/IP 4 rétegű** gyakorlati modell: network access, internet (IP), transport (TCP/UDP), application — ez fut élesben az interneten.
 4. A **router** útválasztási táblája alapján dönt; protokollok: statikus, **distance-vector** (RIP), **link-state** (OSPF, Dijkstra), **BGP** (autonóm rendszerek között).
 5. Az **IPv4** 32 bites cím, hálózat-azonosító + host-azonosító; az **alhálózati maszk** (/24) szabja meg a felosztást; **privát tartományok**: 10.x, 172.16.x, 192.168.x.
 6. Az **IPv6** 128 bites, beépített biztonsággal; a **NAT** privát címeket nyilvánosra fordít — IPv4-cím spórolásra.
 7. A **TCP** kapcsolatorientált, megbízható, stream alapú: **3-way handshake**, ACK-nyugtázás, újraküldés, folyamszabályozás, torlódásvezérlés.
 8. Az **UDP** kapcsolat nélküli, gyors, megbízhatatlan — VoIP, streaming, online játékok, **DNS**, DHCP.
 9. Az **ICMP** hálózati réteg protokoll hibajelzésre és diagnosztikára: Echo Request/Reply (**ping**), Time Exceeded (**traceroute**), Destination Unreachable.
 10. A **DNS** domain név → IP feloldó hierarchikus rendszer (root → TLD → authoritative); rekordtípusok A, AAAA, CNAME, MX, NS, TXT — UDP 53-as porton.
-

10.b Szolgáltatásorientált programozás

1. Az **RPC** távoli eljáráshívás: a kliens stub-bal marshallolja a paramétereket, hálózaton átküldi, szerver oldalon kicsomagolódik és visszatér.
 2. A **gRPC** a Google modern RPC-je: HTTP/2 transzporton, bináris üzenetekkel, 4 streaming-móddal, többnyelvű kódgenerálással .proto fájlokból.
 3. A **protocol buffers** a gRPC szerializációs nyelve — séma-alapú, bináris, kisebb és gyorsabb mint a JSON, és kompatibilis verziókezelést támogat.
 4. A **WEB** decentralizált, HTTP-alapú; az **egyrétegű (monolit)** architektúrában minden egy alkalmazásban van.
 5. A **többrétegű (3-tier)** felosztja a rendszert prezentáció (UI), üzleti logika, adat rétegre — minden réteg külön skálázható.
 6. **Vékony kliens**: kevés a kliensen, sok szerveren (webalkalmazás); **vastag kliens**: sok kliens-oldali logika (asztali alkalmazás, SPA).
 7. **Elosztott rendszerek** skálázhatóak (horizontálisan), hibatűrőek; a **CAP-tétel** szerint a Consistency-Availability-Partition tolerance hármasából csak 2 lehet.
 8. A **REST** HTTP-alapú architektúrális stílus: **resource-orientált URI**, **stateless**, HTTP-igék (GET/POST/PUT/PATCH/DELETE), cache-elhető, JSON.
 9. A **web service** hálózaton elérhető gép-gép API; **SOAP/XML** vagy **REST/JSON** — utóbbi ma az elterjedt.
 10. A **WCF** Microsoft technológia szolgáltatásokhoz, **ABC** modellel (Address, Binding, Contract); az ASMX csak HTTP/SOAP, a WCF rugalmas és sokoldalú, de a modern alternatíva ASP.NET Core Web API vagy gRPC.
-

11.a Rendszerfejlesztés — módszertanok

1. Az **életciklus dokumentumai**: követelményspecifikáció, funkcionális specifikáció, nagyvonalú rendszerterv, teszterv, átadás-átvételi jegyzőkönyv — a funkspec **megfeleltetést** is tartalmaz a kövspechez.
 2. A **módszertanok** oszthatók: életciklus-sorrend (lineáris/iteratív), dokumentáltság (könnyű/nehézsúlyú), megközelítés (jól dokumentált, prototípus, agilis), középpont.
 3. **Strukturált (hagyományos)** módszertanok lineárisak, nehézsúlyúak: **vizesés** nem enged visszatérést, a **V-modell** teszteléssel egészíti ki.
 4. **Hagyományos vs agilis**: a hagyományos sok dokumentációt és merev fázisokat, az agilis működő kódot és rugalmas iterációkat, direkt kommunikációt preferál.
 5. **Prototípus alapú megközelítés**: a felhasználó ne a projekt végén lássa először a terméket — **eldobható** (csak felmérésre) vagy **evolúciós** (a rendszer lelke lesz).
 6. A **Scrum** agilis módszertan, 2-4 hetes **sprintekkel**: Sprint Planning, Daily Meeting, Sprint Review, és a legfontosabb **Retrospective**.
 7. **Scrum szerepekörök**: **Scrum Master** (folyamat-felügyelő), **Product Owner** (priorizál, nem azonos SM-mel), **Team** (5-9 fő); a **Product Backlog** prioritizált sztorikat tartalmaz.
 8. Az **extrém programozás (XP)** 4 tevékenységre (kódolás, tesztelés, odafigyelés, tervezés) és 5 értékre (kommunikáció, egyszerűség, visszacsatolás, bátorság, tisztelet) épül.
 9. **XP technikák**: páros programozás, **TDD**, code review, folyamatos integráció, **refactoring** — akkor jó, ha a megrendelő tud átláthatóan együttműködni.
 10. A **kockázatmenedzsment** két vetülete: bekövetkezés valószínűsége és kár mértéke; a **súlyosság** = **valószínűség × kár**, lépései: azonosítás, értékelés, csökkentés, kommunikáció (COBIT, Common Criteria).
-

11.b Nulladrendű logika

1. A **nulladrendű logika** ítéletváltozókat kapcsol logikai műveletekkel — minden objektum igaz/hamis; a **szintaxis** a helyes formulák nyelvtana, a **szemantika** az értelmezés.
 2. Az **igazságtábla** segít összetett kifejezést kiértékelni; egy **interpretáció** minden atomhoz igaz/hamis értéket rendel.
 3. **Szemantikai tulajdonságok**: **tautológia** (minden interpretációban igaz), **kontradikció** (mindig hamis), **kielégíthető** (legalább egyben igaz).
 4. **Normálformák**: **literál** (atom vagy negáltja), **klóz** (literálok diszjunkciója), **KNF** (klózikonjunkciója), **DNF** (elemi konjunkciók diszjunkciója).
 5. **KNF-re hozás**: ekvivalencia eltüntetése, implikáció $A \rightarrow B \equiv \neg A \vee B$, negációk bevitele atomokig (de Morgan), **disztributivitás**.
 6. A **Tseitin transzformáció** lineáris algoritmus: minden nem-literál részformulához új X ítéletváltozót vezet be, és konjugál hozzá egy $X \leftrightarrow A$ ekvivalenciát.
 7. A **Plaisted-Greenbaum kódolás** csökkenti a Tseitin klózszerát: az ekvivalencia két implikációra bontható, és a **polaritás függvényében** csak az egyik megtartandó.
 8. A **rezolúció** cáfoló eljárás: a tagadásból az **üres klóz** levezetésével UNSAT-ot bizonyítunk — nulladrendűben **véges levezetés garantált**.
 9. A **SAT** (KNF kielégíthetősége) **NP-teljes**; **3-SAT** is, **2-SAT** polinomiális; a **DPLL** visszalépéses keresés + **unit propagáció** az egységklózikra.
 10. A **DIMACS** a SAT solver standard input formátuma (sorszámok, **-** negálás, **0** klózvég); az **SMT** kiterjeszti aritmetikára/tömbökre/listákra, az **SMT-LIB** ennek emberközelibb formátuma.
-

12.a Adatbázis II — PL/SQL

1. A **PL/SQL** az Oracle procedurális kiterjesztése — változókkal, vezérléssel, kivételkezeléssel ágyazza be az SQL-utasításokat.
 2. **Típusok**: skalár (NUMBER, VARCHAR2, DATE, BOOLEAN); **%TYPE** (oszlop típusa), **%ROWTYPE** (sor típusa), RECORD, TABLE (asszociatív tömb), VARRAY.
 3. Változó deklarálása **név típus := kezdőérték** ; **CONSTANT** konstanst, **NOT NULL** kötelező értéket ad.
 4. **Vezérlési szerkezetek**: **IF/ELSIF/ELSE** , **CASE** , **LOOP-EXIT WHEN** , **WHILE** , **FOR i IN 1..10** , és cursor-FOR-loop ami automatikusan kezeli a kurzort.
 5. **SQL beágyazása**: **SELECT INTO** egy sort egy változóba (NO_DATA_FOUND, TOO_MANY_ROWS); INSERT/UPDATE/DELETE közvetlenül; DDL csak **EXECUTE IMMEDIATE** -tel.
 6. Egy **PL/SQL blokk** 3 részből áll: **DECLARE** (deklarációk), **BEGIN-END** (kötelező), **EXCEPTION** (opcionális) — blokkok egymásba ágyazhatók.
 7. A **tárolt eljárás (PROCEDURE)** nem ad vissza értéket (csak OUT-on), önállóan hívható; a **tárolt függvény (FUNCTION)** **RETURN** -nel ad értéket és SELECT-ben is hívható.
 8. A **csomag (PACKAGE)** logikailag összetartozó alprogramokat csoportosít: **specifikáció** (interface) és **body** (implementáció), encapsulation, memóriába egészében töltődik.
 9. **Paraméter-módok**: **IN** (alapeset, csak olvasható), **OUT** (kimenet, null-ról indul), **IN OUT** (be- és kimenet) — pozicionális vagy megnevezett (**a ⇒ 1**) hívás.
 10. **Kivételkezelés**: **WHEN** beépített kivétel (NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, OTHERS) vagy saját **EXCEPTION** + **RAISE** ; **RAISE_APPLICATION_ERROR** -20000-től -20999-ig.
-

12.b Számítógépi grafika

1. A **DDA szakaszrajzoló** az egyenes $y = mx + b$ alakjából lépésekkel rajzolja a pixeleket, de lebegőpontos + kerekítés miatt nem ideális.
 2. A **MidPoint szakasz** csak egész számokkal: az aktuális pont után a **felezőpont** alapján dönt East vagy NorthEast pixelről.
 3. A **MidPoint körrajzoló** kihasználja a **nyolcas szimmetriát** (elég a kör 1/8-át kiszámolni), és a felezőpont alapján E vagy SE pixelt választ.
 4. A **Cohen-Sutherland vágó** algoritmus 4 bites kódot rendel a sík 9 részéhez; 0000 kódok → belül, $AND \neq 0$ → kívül, egyébként élvonalakkal vágunk.
 5. A **Hermite-ív** két végpont és két érintővektor alapján harmadfokú polinom — négy Hermite-polinom súlyozott kombinációja.
 6. A **Bézier-görbe** approximáció (csak végpont-interpoláció), **Bernstein-polinommal** előállítva — **konvex burok** tulajdonság, affin invariancia, magas fokszámnál merev.
 7. A **B-Spline** lokálisan hat (új kontrollpont nem mozgatja az egész görbét), automatikus folytonossággal csatlakoznak az ívek.
 8. A **homogén koordináták** egy 3D pontot $(x, y, z, 1)$ alakkal reprezentálnak — így az **eltolás is mátrixszorzással** leírható, **4×4 mátrixokkal**.
 9. **Pont-transzformációk**: eltolás, forgatás, tükrözés, skálázás, nyírás; **vetítések**: párhuzamos (megtartja arányokat), centrális (perspektív, realisztikus), **axonometria**.
 10. Poliédereket **Wire Frame** (csak csúcs+él) vagy **B-Rep** (geometriai+topológiai) tárol; megjelenítés: hátsó lapok eltávolítása normállal, **Z-Buffer**; árnyalás: **Flat** (lapszín), **Gouraud** (csúcs-szín interp.), **Phong** (normál interp., legszebb).
-

13.a Webprogramozás II — PHP

1. A **statikus weboldal** mindig ugyanazt küldi; a **dinamikus** futás közben generálja a tartalmat felhasználó/adatbázis alapján (PHP, Python, Node.js).
 2. A **PHP** szerver-oldali, dinamikusan típusos, interpretált nyelv — C-szerű szintaxis, HTML-be ágyazható (WordPress, Drupal).
 3. A futáshoz **webszerver** (Apache/Nginx) és **PHP interpreter** kell — a klasszikus **LAMP stack**: Linux + Apache + MySQL + PHP.
 4. **Típusrendszer**: skalár (int, float, string, bool), összetett (array, object, callable), speciális (null, resource) — a típus futás közben változhat.
 5. A változó `$` prefixxel, a **konstans** `define()` -al vagy `const` -tal; **superglobálisok**: `$_GET` , `$_POST` , `$_SESSION` , `$_COOKIE` , `$_FILES` , `$_SERVER` .
 6. **HTML-PHP kapcsolat**: `<?php ?>` blokkokkal — a PHP feldolgozza a kódot, a többbit változatlanul küldi a kliensnek.
 7. **Modulszerkezet**: `include` / `require` (utóbbi fatális hiba hiányzó fájlnál), `*_once` (csak egyszer), **PSR-4 autoloading** modern projekteknel.
 8. **Adatcsere**: GET URL-ben (látható, cache-elhető), POST body-ban (rejtett), **AJAX** aszinkron a böngészőből, **JSON** az API-knál.
 9. **Biztonsági kérdések**: SQL Injection ellen **prepared statement**; XSS ellen `htmlspecialchars` ; CSRF ellen token + SameSite cookie; jelszó `password_hash` / `password_verify` .
 10. A **süti (cookie)** kliensen tárolt 4 KB-ig terjedő adat (HttpOnly, Secure, SameSite flag-ekkel); a **session** szerveroldali állapot, csak a session ID megy cookie-ban.
-

13.b Programozási technológiák — tervezési minták

1. A **tervezési minta** újrahasznosítható megoldás egy gyakori problémára — közös szókincs, bevált megoldás, karbantarthatóság; a **Gang of Four** 23 mintát rendszerezett.
 2. **Három fő kategória: létrehozási** (objektum-példányosítás), **szerkezeti** (összeszerelés), **viselkedési** (kommunikáció).
 3. A **Stratégia** minta interface-t és cserélhető implementációkat ad, a kontextus **kompozícióval** tartja a stratégiát — futás közben váltható.
 4. A **Sablon metódus** minta absztrakt osztály template metódusával adja az algoritmus vázát; a változó lépéseket absztrakt metódusként hagyja — **öröklődéssel** működik.
 5. A Stratégia és a Sablon metódus különbsége: az előbbi **kompozíció + futás-közbeni csere**, az utóbbi **öröklődés + fordításkori rögzítés**.
 6. A **Megfigyelő (Observer)** egy alany állapotváltozásairól értesít minden megfigyelőt anélkül, hogy ismerné őket — fajtái **push** (adatot is ad) és **pull** (csak értesít).
 7. A **SOLID** alapelvek: Single Responsibility, **Open/Closed**, Liskov Substitution, Interface Segregation, Dependency Inversion.
 8. A **nyitva-zárt alapelv** szerint a kód bővítésre nyitott, módosításra zárt — új viselkedés új osztály, a meglévőt nem nyúljuk; **interface + polimorfizmus** a kulcs.
 9. **GOF1**: programozz interfészre, ne implementációra; **GOF2**: részesítsd előnyben a kompozíciót az öröklődéssel szemben.
 10. A **TDD három lépése** Red-Green-Refactor: bukó teszt, minimális kód a zöldhöz, majd szépítés — kiegészíti a code review, CI/CD, refactoring, naming conventions.
-

14.a Programozási technológiák — OOP elvek + minták

1. Az **osztály felület** (mit kínál kívülről) és **megvalósítás** (hogyan oldja meg) két részből áll — a felület stabil, a megvalósítás cserélhető.
 2. Az **objektum három jellemzője**: felület (publikus metódusai), **belső állapot** (privát mezők értéke), **viselkedés** (hogyan reagál a hívásokra).
 3. Az **OOP négy alapelve** a fenti fogalmakkal: egységbe záras (adat + művelet együtt), adatrejtés (állapot csak felületen át), öröklődés (felület + megvalósítás), polimorfizmus (azonos felület, eltérő viselkedés).
 4. A **GOF1** alapelv interface-re programozást ír elő — a változó típusa legyen a legabsztraktabb, így a konkrét osztály cserélhető.
 5. A **GOF2** alapelv a **kompozíciót** részesíti előnyben az öröklődéssel szemben — futás közben cserélhető, lazább kapcsolat, mint az "is-a".
 6. A **Stratégia minta** közös interface + kontextust ad, ami kompozícióval tartja a stratégia-példányt; `SetStrategy()` metódussal cserélhető — **nyitva-zárt** elv teljesül.
 7. **Viselkedési minták általában**: Strategy, Template Method, Observer, Iterator, Command, State, Chain of Responsibility, Mediator, Visitor.
 8. Az **Egyke (Singleton)** garantálja az egyetlen példányt: **privát konstruktor**, **static Instance property**, lazy init lock-kal (vagy `Lazy<T>`) — logger, konfiguráció, connection pool.
 9. **Létrehozási minták általában**: Singleton, Factory Method, Abstract Factory, Builder, Prototype.
 10. A **Díszítő (Decorator)** ugyanazt az interface-t implementálja, mint a díszítendő, körbeveszi, és funkciót ad hozzá (Compress, Encrypt) — **szerkezeti minták**: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy.
-

14.b Adatbázis II — kurzor + trigger + kollekció

1. A **kurzor** a memóriában lévő környezeti terület azonosítója, ami az SQL eredményét tárolja és mutatóval halad a sorokon.
2. **Implicit kurzor** automatikusan jön létre DML-utasításokhoz és 1 sort visszaadó SELECT-hez; attribútumai `SQL%FOUND`, `SQL%NOTFOUND`, `SQL%ROWCOUNT`, `SQL%ISOPEN` (mindig FALSE).
3. **Explicit kurzor 4 lépése:** `DECLARE` (`CURSOR név IS ...`), `OPEN` (lefuttatja a lekérdezést), `FETCH` (egy sort betölt), `CLOSE` (felszabadít).
4. Az **egyszerűsített FOR-LOOP** automatikusan kezeli az open-fetch-close-t: `FOR rec IN (SELECT ... ) LOOP ... END LOOP`.
5. **Explicit kurzor-attribútumok:** `%FOUND` (sikeres FETCH után TRUE), `%NOTFOUND`, `%ISOPEN`, `%ROWCOUNT` (eddig betöltött sorok); nincs megnyitva → `INVALID_CURSOR` kivétel.
6. A **kurzorváltozó (REF CURSOR)** dinamikus — futás közben bármely kompatibilis lekérdezéshez kapcsolható; **erős** (RETURN típussal) vagy **gyenge** (típus nélkül).
7. **Tranzakciókezelés:** `COMMIT` véglegesít, `ROLLBACK` visszagörget, `SAVEPOINT` köztes mentés; DDL implicit COMMIT-tal jár.
8. A **trigger** automatikusan végrehajtódó kód egy eseményre (tábla módosulás, rendszer); típusai **sor-** vs **utasításszint**, **BEFORE/AFTER/INSTEAD OF**, rendszer-trigger.
9. **Trigger-felhasználás:** naplózás, integritás, származtatott érték frissítése, üzleti szabály — sor-szintű triggerben `:NEW` és `:OLD` érhető el.
10. **Kollekciók 3 típusa:** **asszociatív tömb** (hash, csak PL/SQL, int/string index), **beágyazott tábla** (DB oszlop is lehet, lyukak, törölhető elem), **VARRAY** (fix max méret, folytonos, nem törölhető elem).